PICO: Ordering-Centric Cost-Based Optimization for ML Input Pipelines

谢瑞阳

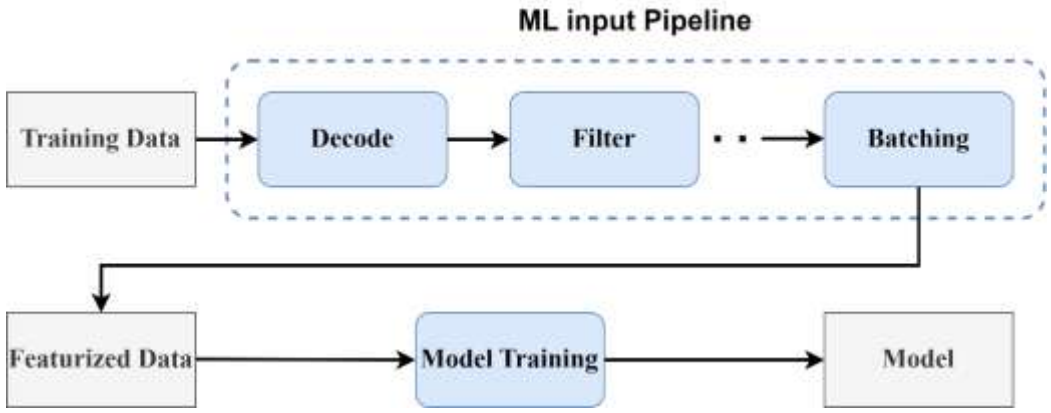
華東師範大學

Outline

1. Background
2. Motivation
3. Optimal Reordering
4. Joint optimization
5. Experiment

ML input pipeline

An ML input pipeline performs the online, last-mile processing between stored data and the training loop.

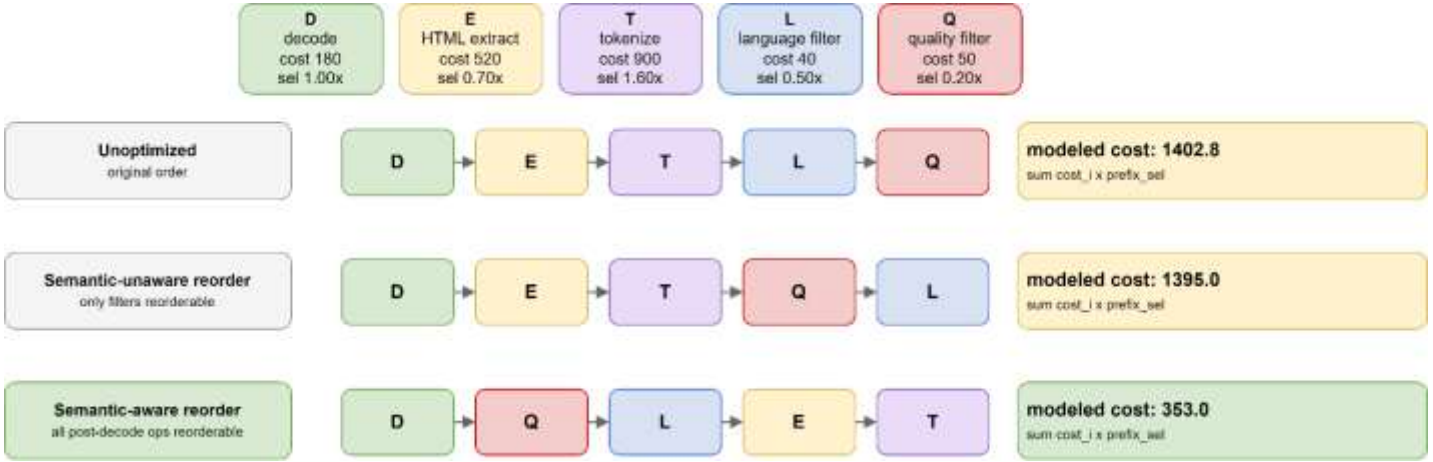


Optimization Opportunities in Input Pipelines

- Prior input systems improve throughput with techniques such as parallelism, prefetching, caching, offloading, fusion, and operator reordering.
  - Operator reordering changes the logical order in which transformations are applied.
  - Caching materializes the output of a deterministic prefix.
  - Offloading assigns an operator or operator block to an execution backend.
  - Fusion combines adjacent compatible operators into one physical unit.

Semantic-Aware Operator Reordering

- Semantic-aware reordering uses dependency and randomness annotations to identify which operator orders are fixed and which can be safely changed.
- This enables the optimizer to explore profitable reorderings without relying on relational algebra assumptions or risking invalid transformations.



Cost-Based Input Pipeline Optimization

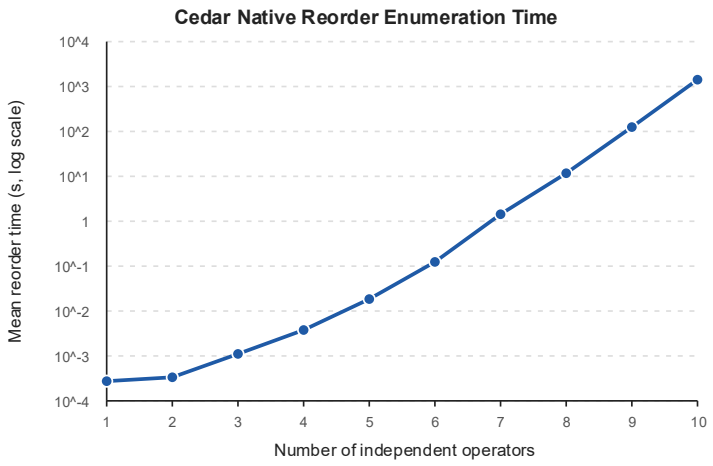
- A cost-based optimizer provides a common basis for comparing these heterogeneous decisions.
- It first profiles the pipeline to estimate operator cost and selectivity, then uses these measurements to assign a modeled cost to each legal physical pipeline.
- Cedar is the representative cost-based optimizer for this setting.

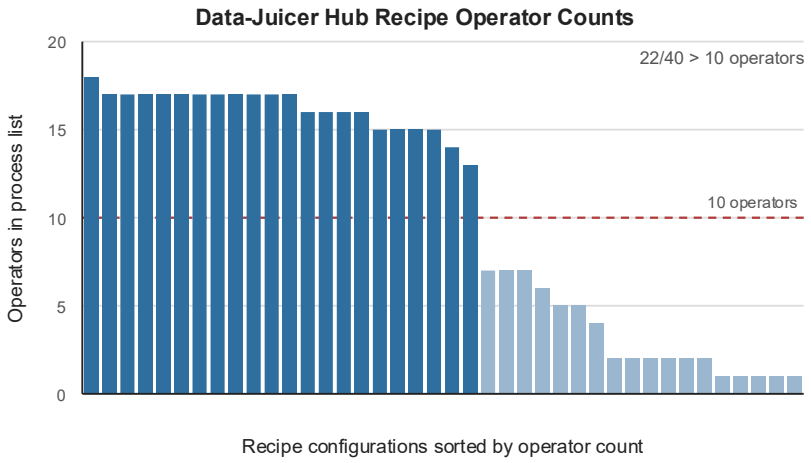
Outline

1. Background
2. Motivation
3. Optimal Reordering
4. Joint optimization
5. Experiment

Semantic-Aware Operator Reordering Overhead

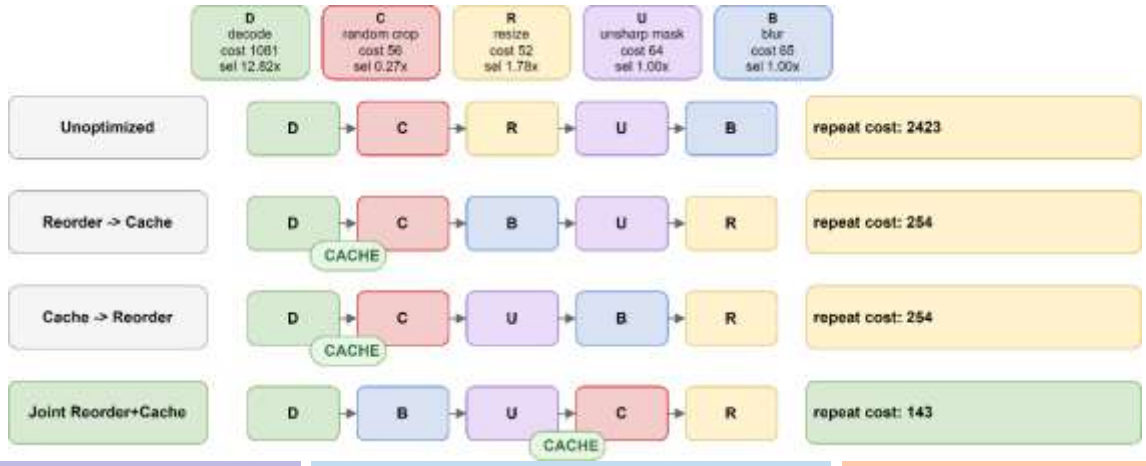
- Semantic-aware reordering search over all legal operator orders. Even with dependency constraints, the number of possible orders can grow factorially.
- This becomes expensive as input pipelines contain more reorderable operators.





Co-Optimization in Input Pipelines

- Input pipeline optimizations are tightly coupled.
- Stage-wise optimization may commit to a locally good plan too early.
- Joint optimization keeps these choices in the same search space, allowing the optimizer to find globally better pipeline plans.



Co-Optimization in Input Pipelines

- 虽然联合优化很好，但由于语义感知重排本身的开销较大，即使面向比较小的pipeline，我们也无法将其与其他优化技术做联合优化，因为这会导致较高的优化开销。

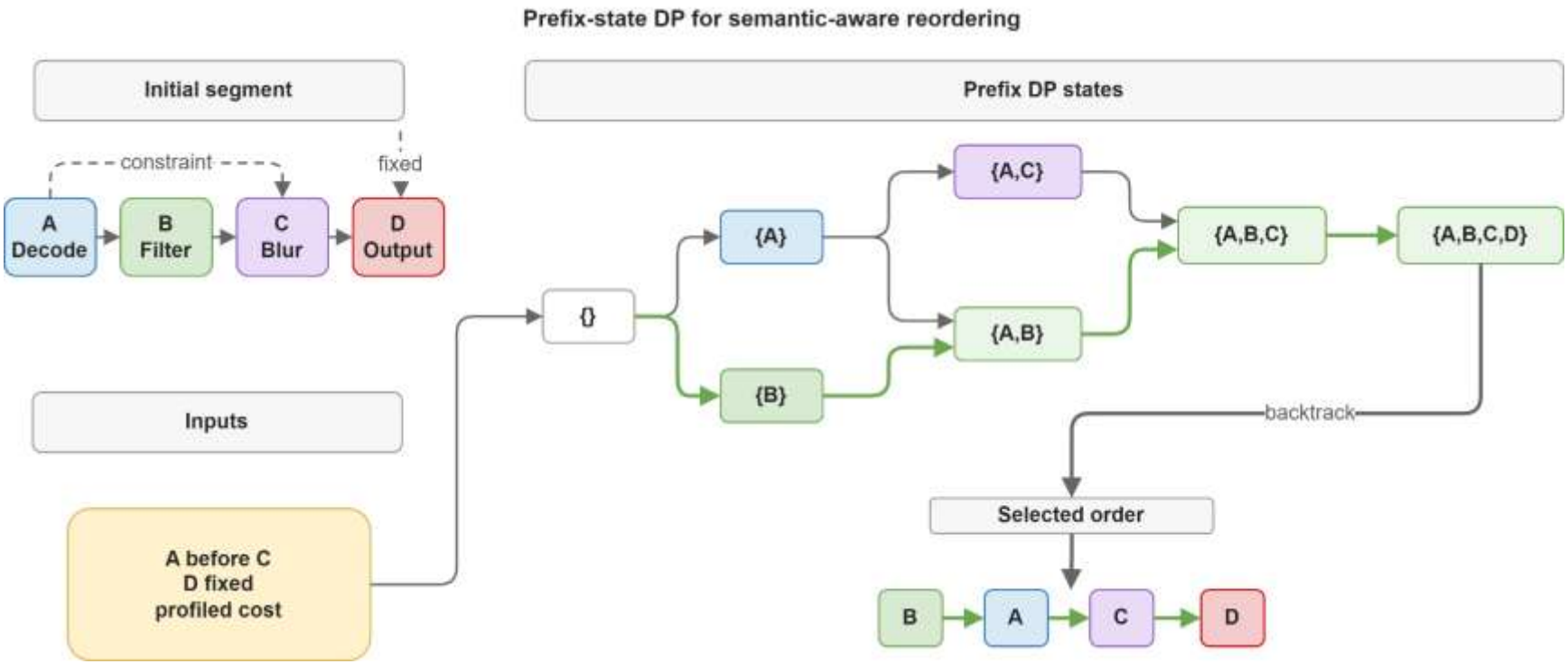
Outline

1. Background
2. Motivation
3. Optimal Reordering
4. Joint optimization
5. Experiment

Opportunity: Prefix-State DP

- Many candidate orders share the same intermediate prefixes.
- Instead of enumerating every complete order, PICO represents search by the set of operators already placed.
- Each prefix state is evaluated once and reused by later extensions.

Prefix-State Dynamic Program

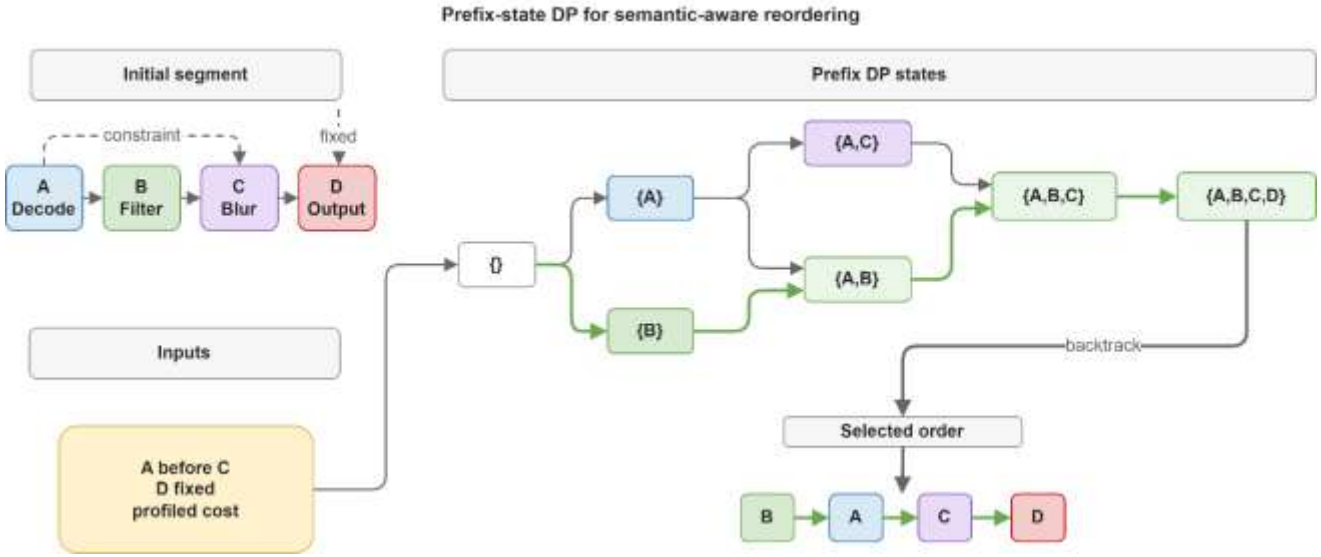


Reordering Problem Setup

- PICO optimizes reorderable linear segments in an input pipeline.
- Fixed boundaries and semantic dependencies define the legal search space.
- Profiling provides operator costs, input sizes, and data-volume ratios.
- The goal is to find the lowest-cost legal order under the profiled cost model.

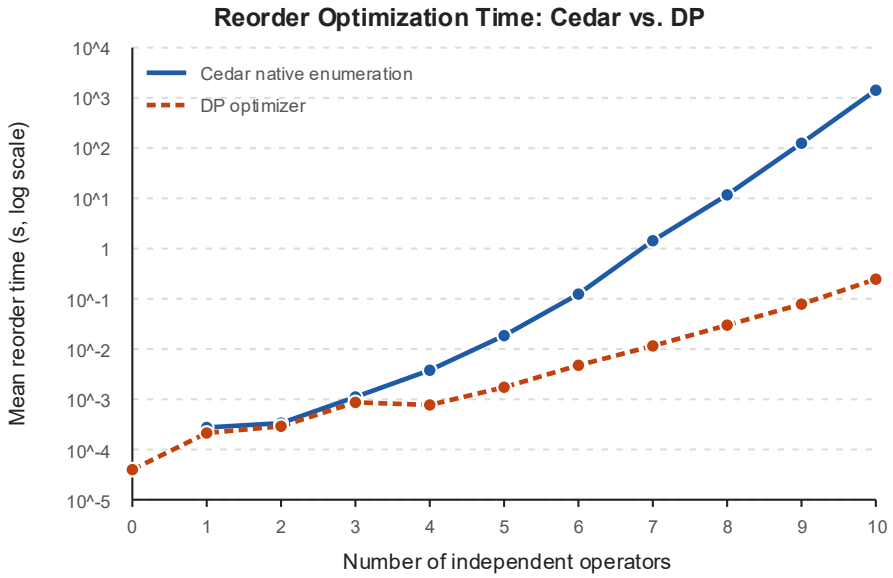
Prefix-State Dynamic Program

- Many candidate orders share the same set of already placed operators. Under the cost model, this set determines future data volume and legal next choices.
- PICO keeps only the cheapest prefix for each operator set. Backpointers reconstruct the final minimum-cost legal order.



Properties

- PICO searches subset states instead of complete permutations.Reordering takes O(n * 2^n) time, avoiding factorial enumeration.
- The optimality guarantee is scoped to the given semantic constraints and profiled cost model.



Outline

1. Background
2. Motivation
3. Optimal Reordering
4. Joint optimization
5. Experiment

Integrated DP View

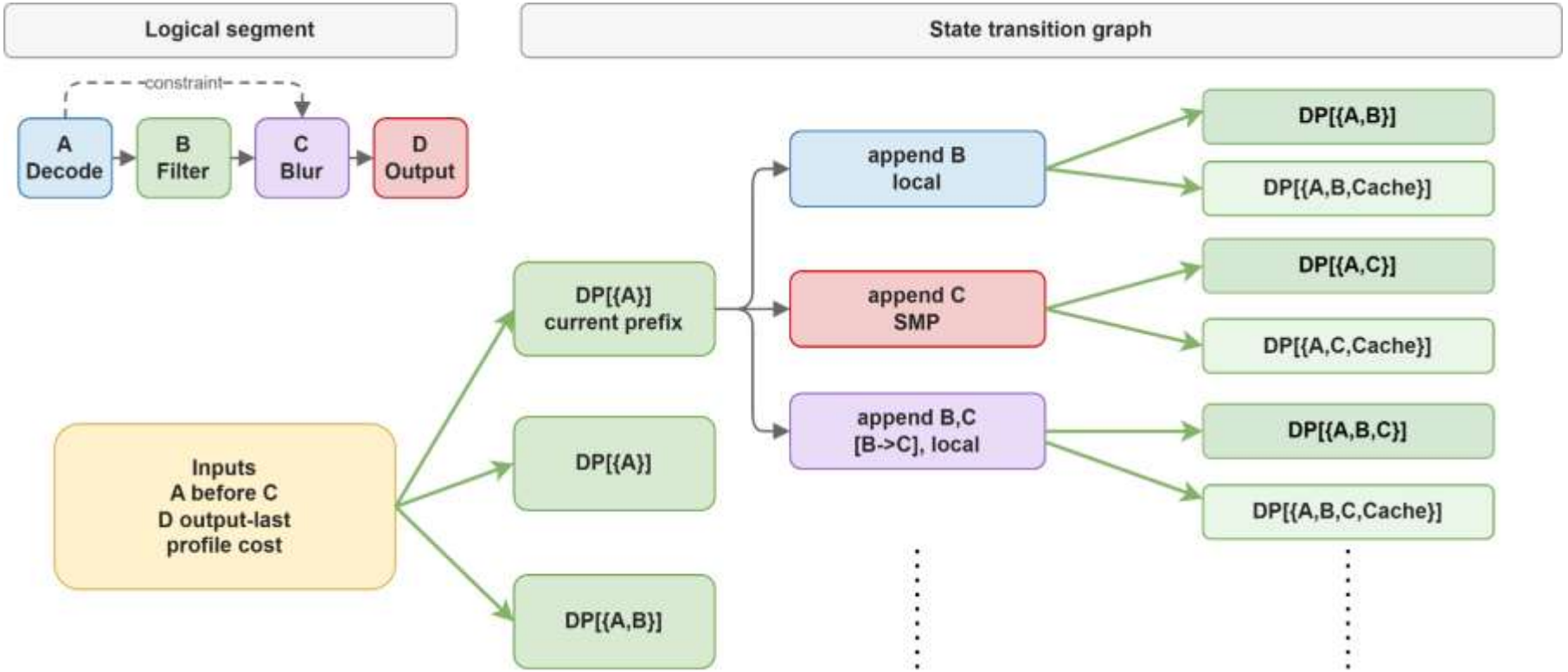
- PICO extends prefix-state DP from logical reordering to physical pipeline optimization.
- Each transition represents one physical planning decision.
- A transition may append a singleton operator or fused block, select a physical variant, and update cache state.
- Reordering, caching, offloading, and fusion are compared in the same DP.

Transition Candidates

- Before the outer DP, PICO precomputes feasible block candidates.
- For each variant, an inner DP finds the minimum-cost internal order for every block.
- PICO then compares feasible variants for the same block and keeps the lowest-cost implementation.
- The outer DP consumes each candidate as a fixed block with a known order, variant, and cost.

Transition Candidates

- The outer DP consumes each candidate as a fixed block with a known order, variant, and cost.

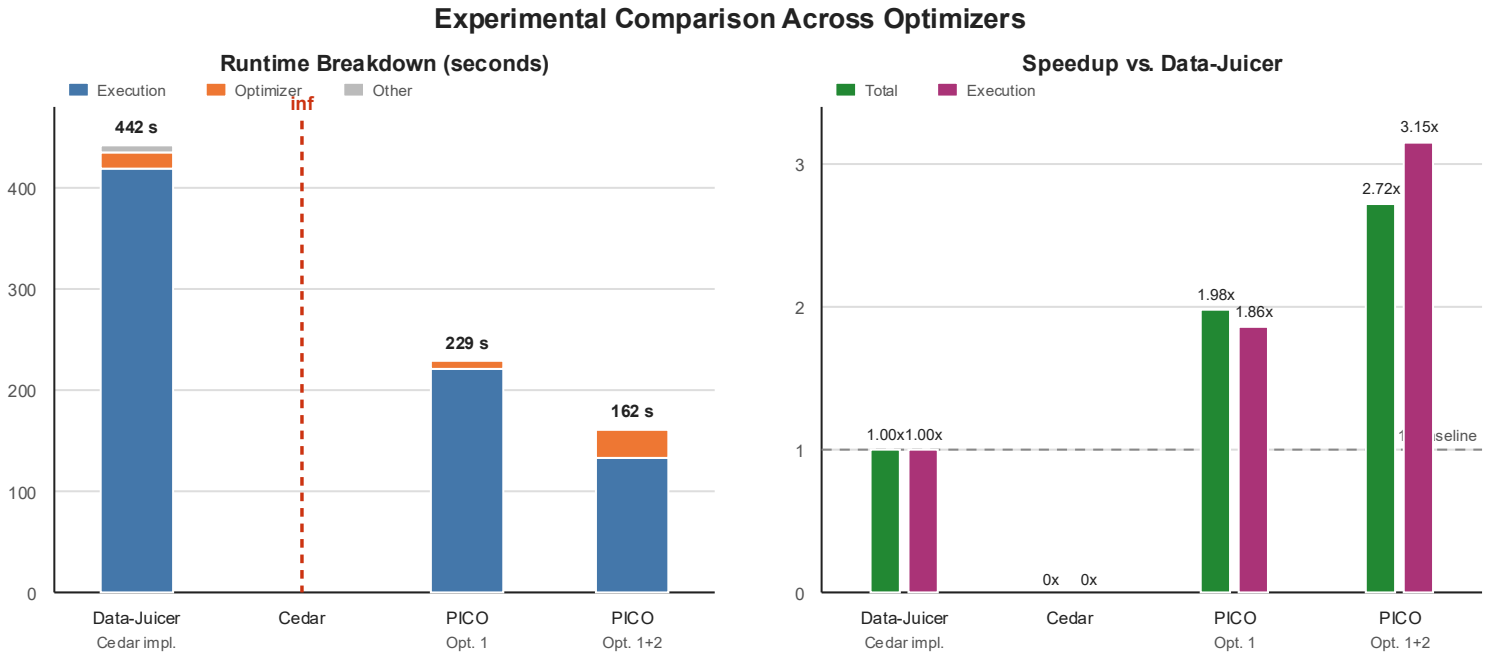


Outline

1. Background
2. Motivation
3. Optimal Reordering
4. Joint optimization
5. Experiment

Experiment

- Bloom_Oscar workload with Wikitext03 dataset.



Thanks!